Contents

RAG Foundations

Understand the complete retrieval pipeline, from raw documents to a grounded answer, before reaching for a framework.

`Theory + raw Python` `OpenAI embeddings` `Qdrant`

---

part one

What RAG is, and why it exists

The big picture

Before any detail, hold this one picture in your head. Every RAG system, from a weekend project to an enterprise search engine, is two pipelines joined by a database.

Indexing Documents → Parsing → Chunking → Embedding → Ingest into Qdrant

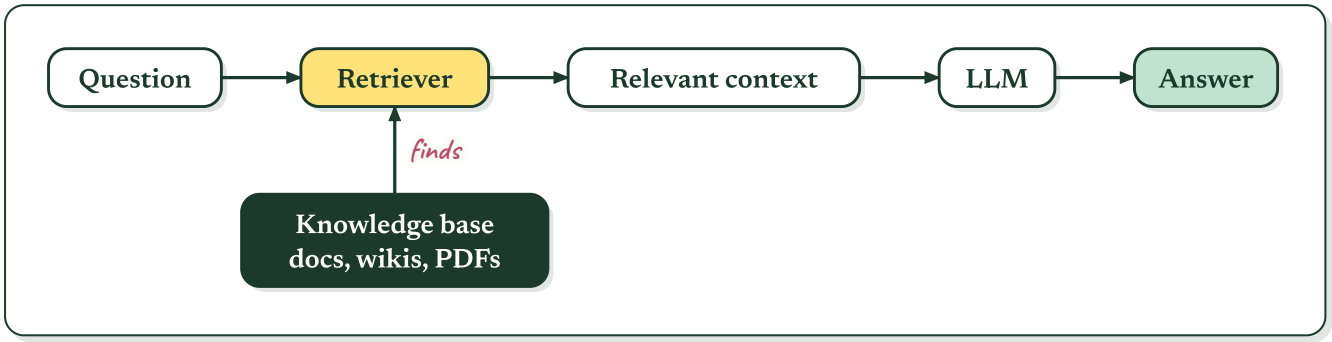
Querying User query → Query embedding → Similarity search → Relevant chunks → LLM → Answer

Indexing happens ahead of time, once per document. Querying happens live, once per question. Every arrow in both rows is explained in the sections that follow.

> *Go deeper* Pinecone: RAG overview Microsoft: Design and develop a RAG solution

What is RAG?

Retrieval-Augmented Generation gives a language model relevant information, retrieved from an external knowledge source, *before* it writes an answer. The idea was formalised in Lewis et al., 2020.



> *The retrieval system finds information. The LLM generates the response.*
>
> Keep these two jobs separate in your head. When an answer is wrong, the first question is always: did we retrieve the right thing?

Where RAG is used

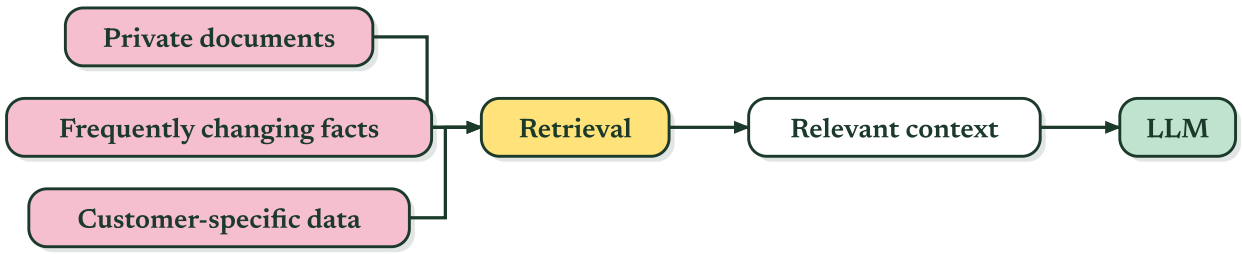
Use case	Example
Internal knowledge	Company policies and handbooks
Support	Product troubleshooting
Documentation	API and product docs
Legal	Contract clauses
Finance	Reports and filings
Education	Course and book content

Use case	Example
Code	Repository documentation

Why RAG is needed

An LLM's parameters are a poor database. They were frozen at training time, they mix facts with style, and you cannot point to where an answer came from. They are especially unreliable for:

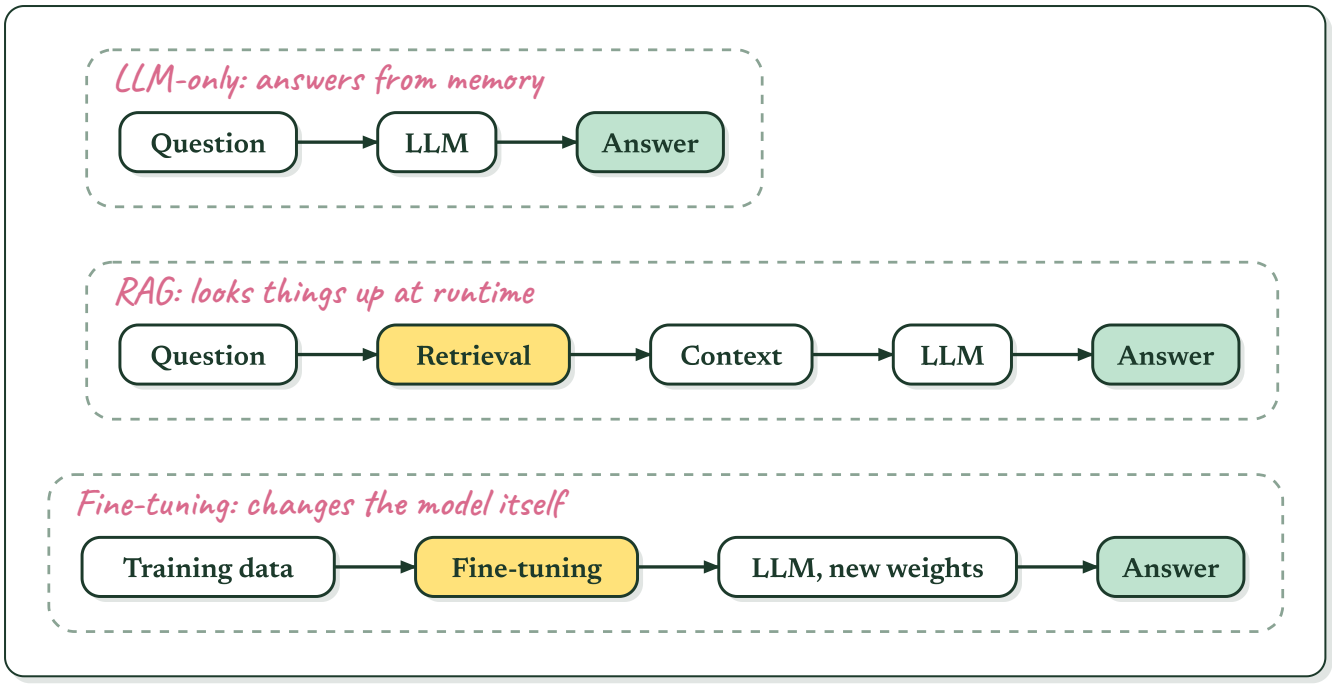
- Private information
- Frequently changing information
- Internal documentation
- Customer-specific data
- Large proprietary document collections



watch out

RAG improves grounding. It does **not** guarantee correct answers. Bad retrieval still produces a bad answer, just a more confident-looking one.

RAG vs LLM-only vs fine-tuning



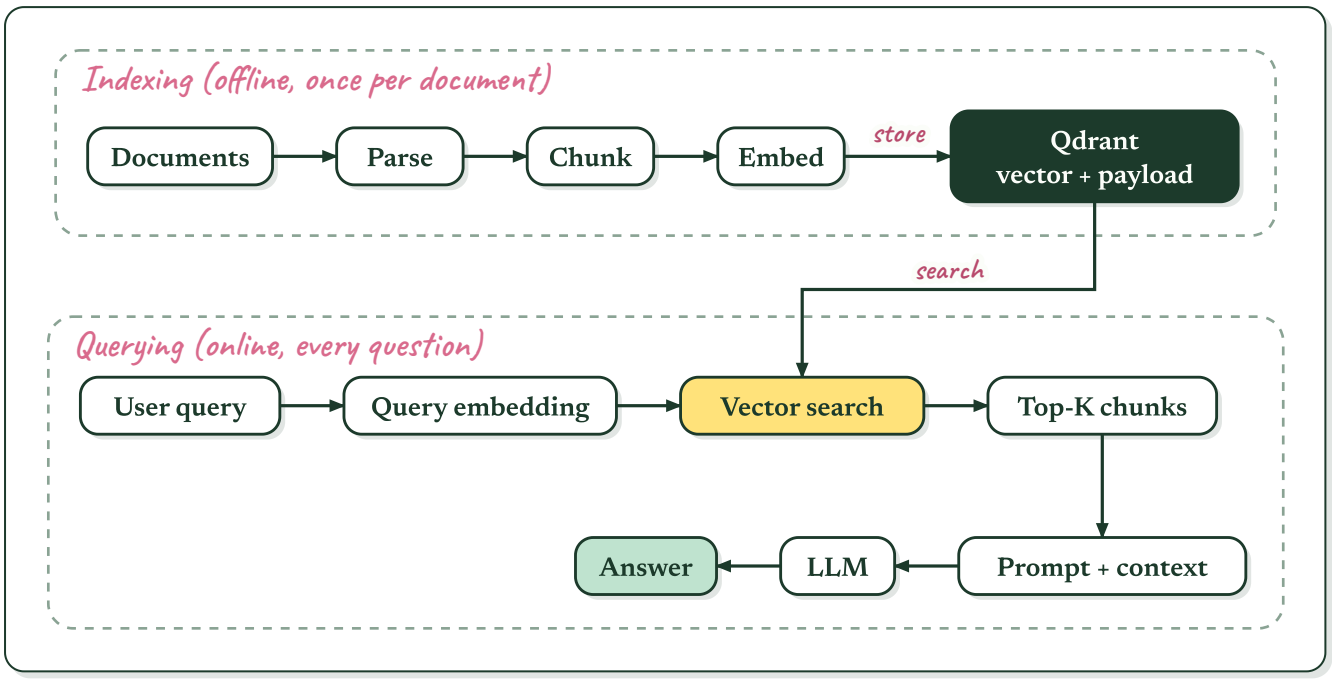
Fine-tuning is a different tool. It changes the model's *parameters*. RAG leaves the model alone and changes what it *sees* at question time.

	RAG	**Fine-tuning**
Best for	External, current knowledge	Behavior and task adaptation
Updating data	Update the index, independent of the model	Needs a training process for every change
Quality depends on	Retrieval quality	Training-data quality

> *"The model needs access to this information" → think RAG.*
> *"The model should behave differently" → fine-tuning may be relevant.*

Basic RAG architecture

Here is the whole system as one diagram. The vector database sits in the middle: indexing writes into it, querying reads from it.



tip

Each stage builds on the one before it: RAG, indexing vs querying, documents, chunking, embeddings, similarity, then vector databases. Frameworks such as LangChain wrap these same stages, so understanding the mechanics first makes their abstractions easy to reason about.

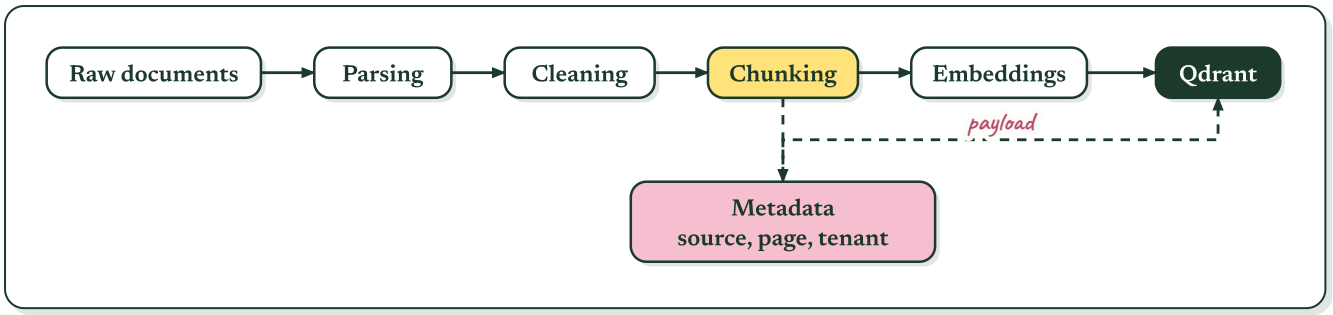
---

part two

Getting documents ready

Document ingestion pipeline

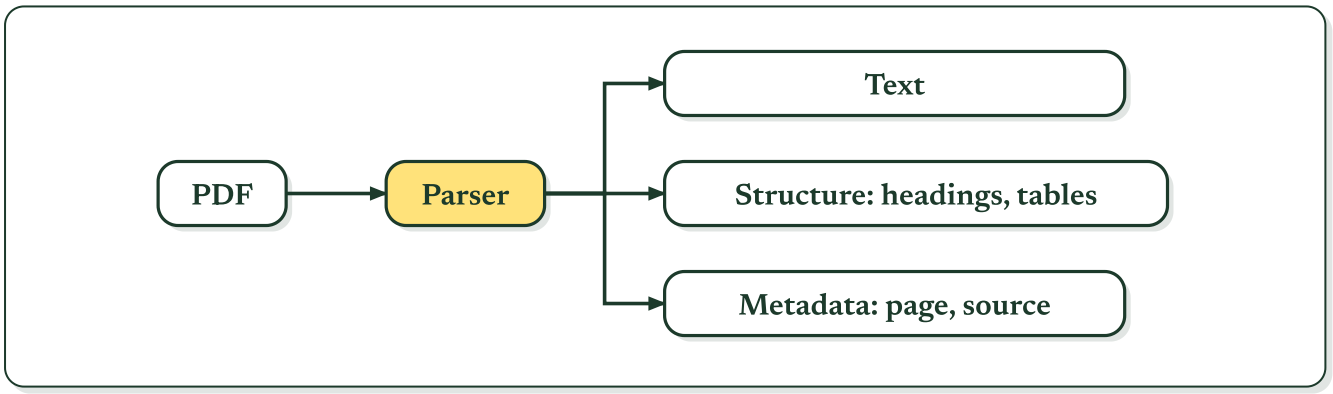
Ingestion is the indexing half of RAG: turning messy raw files into searchable records. Each stage hands its output to the next.



Treat it as a pipeline you can inspect stage by stage. Microsoft's guide recommends evaluating each stage separately, because a failure early on is invisible by the time you read the final answer.

Document parsing

Knowledge hides in many formats: PDF, DOCX, HTML, Markdown, TXT, email, JSON, databases and source code. Parsing converts each one into usable content.



Parsing is an engineering problem in its own right. A PDF can contain tables, multiple columns, headers, footers and a broken reading order. If the parser reads two columns as one, every later stage inherits the damage.

Bad parsing → Bad chunks → Bad embeddings → Bad retrieval → Bad RAG

tip

Habit to build: always print a few parsed pages and read them with your own eyes before chunking anything.

Metadata

A chunk of text alone is not enough. Attach information about *where it came from* and *who may see it*:

```
{
  "text": "Employees receive 20 days of annual leave.",
  "source": "employee-handbook.pdf",
  "page": 14,
  "department": "HR",
  "tenant_id": "company-123"
}
```

Copy

This is what makes the following possible:

Filtering
HR docs only

Citations
"see page 14"

Source tracking
which file?

Multi-tenancy
company-123 only

Access control
who may read

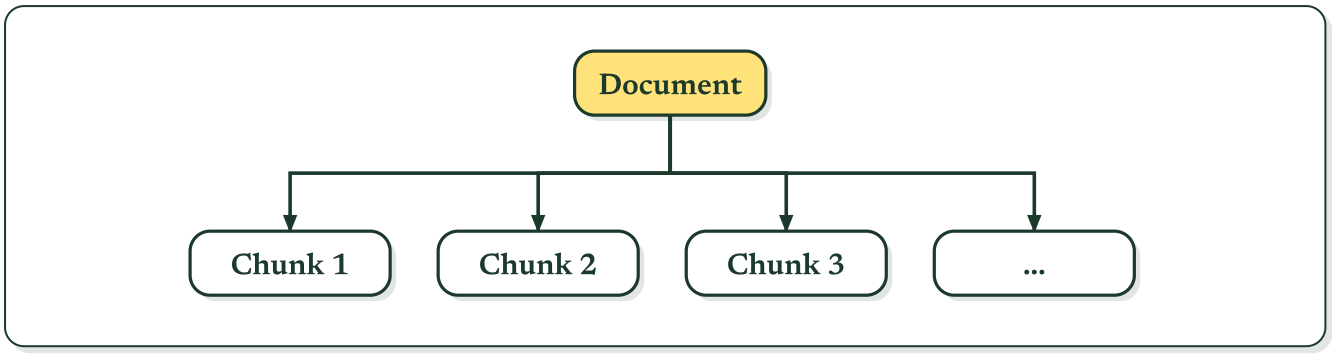
Debugging
why this chunk?

Qdrant calls this extra information the payload.

Chunking

A 200-page manual cannot be one vector, and it should not be sent to the LLM whole. We split documents into smaller retrieval units called **chunks**, and every chunk gets its own

vector.



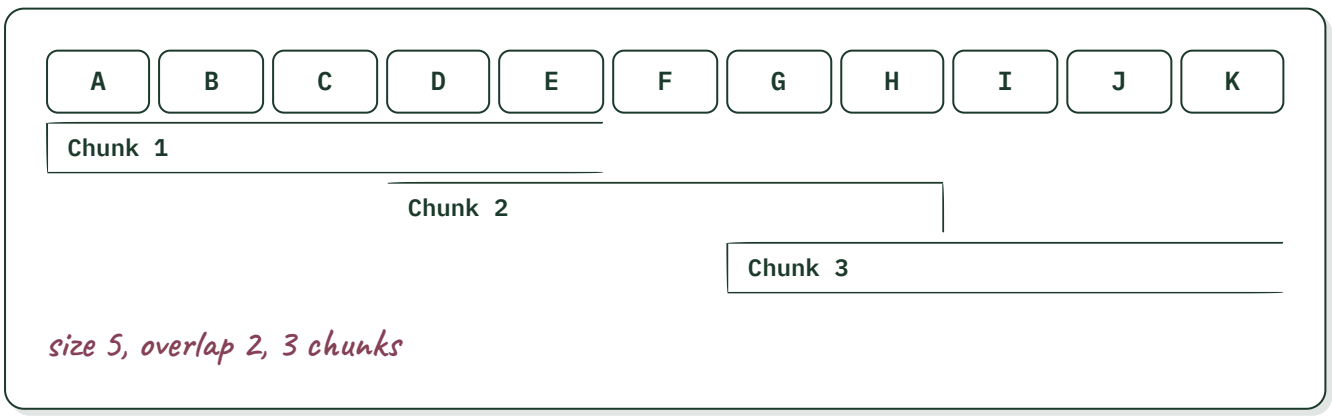
The goal: retrieve a chunk that holds **enough context to answer the question** without dragging along a lot of irrelevant text. The simplest way to get started is to cut the text into windows of fixed size, and let neighbouring windows **overlap** a little so that a sentence on a boundary is not lost.

```
def chunk_text(text, size=500, overlap=100):
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]

chunks = chunk_text(open("handbook.txt").read())
print(len(chunks), "chunks")
print(chunks[0][:200])
```

Copy

With a window of 5 and an overlap of 2, each chunk repeats the last two pieces of the one before it:



Too small

Too large

The answer is split across chunks that individually mean little, so retrieval misses it.

The matching sentence is buried in noise, and you pay for every extra token sent to the LLM.

note

There is no universal best chunk size, so start simple, measure, and adjust. Production systems use smarter strategies, such as splitting on headings and paragraphs, counting tokens, or cutting where the topic changes, but fixed windows with a little overlap are the foundation they all build on.

Go deeper Microsoft: the chunking phase Pinecone: chunking strategies

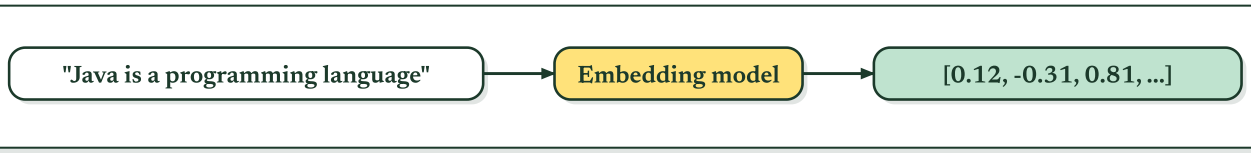
---

part three

Turning text into numbers

Text to vector

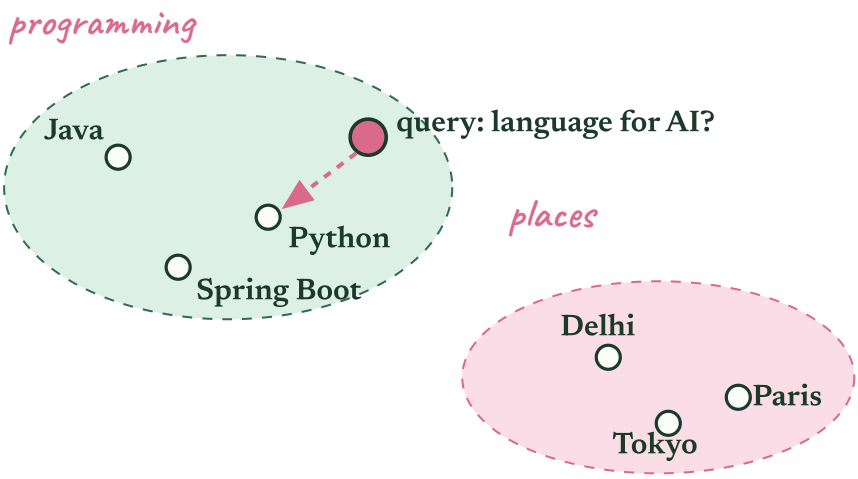
An **embedding** is a numerical representation of an object. For us the object is text, and the representation is a list of numbers called a vector.



1536 numbers [`0.12` `-0.31` `0.81` `0.04` `-0.57` `0.22` `0.09` `-0.44` …]

The model maps text into a mathematical space where relationships can be *measured*. Texts with similar meaning land near each other. That single property is the engine of semantic

search. Qdrant's vector documentation and Jay Alammar's visual Illustrated Word2vec are good companions.



A 2D sketch of meaning space. Real embeddings have 1536 dimensions, but the idea is the same: related things sit close together.

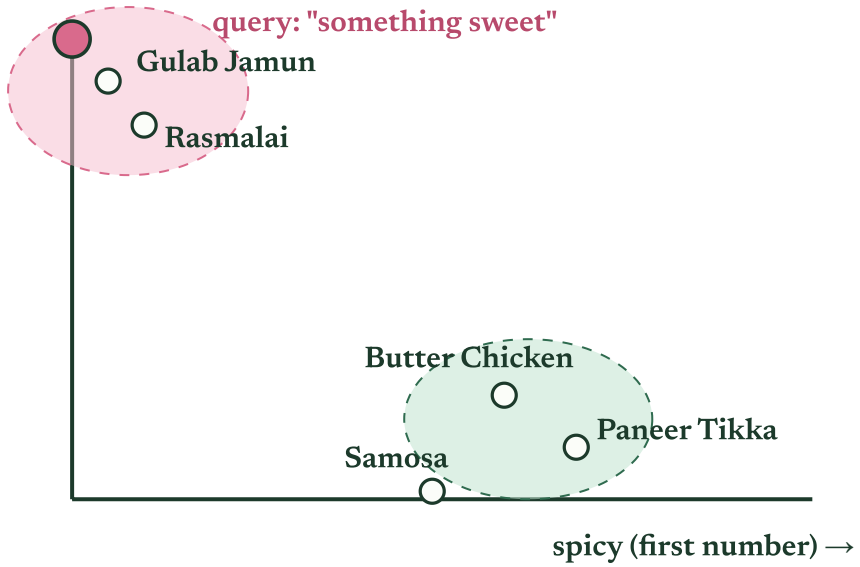
What is a dimension?

This is the central idea behind embeddings, so it is worth building up slowly. A **dimension** is simply *one number that locates a point along one axis*. The number of dimensions is how many numbers you need to say exactly where something is.

One, two and three numbers

- **1 dimension:** a position on a ruler. One number is enough: `7 cm`.
- **2 dimensions:** a spot on a map. You need two numbers: latitude and longitude.
- **3 dimensions:** a drone in the air needs three: latitude, longitude and height.

Now use the same idea for food. Suppose we describe every dish with two scores between 0 and 1: how **spicy** it is and how **sweet** it is. Each dish becomes a point on a plot, and its two scores are its coordinates.



Two numbers per dish give a 2-dimensional space. Dishes that taste alike end up close together.

Gulab Jamun [`0.05` `0.95`]

Butter Chicken [`0.60` `0.20`]

The two numbers in each strip are the dish's **2-dimensional vector**. If we also scored how *crunchy* a dish is, we would have three numbers, a 3-dimensional vector, and a third axis coming out of the page. Samosa would shoot up along it. There is no limit to how many properties we can add.

Now the leap to embeddings

An embedding does exactly this, with two big differences:

1. Far more numbers

Instead of 2 or 3 scores, a text gets **384, 768, 1536 or 3072** numbers. A vector with 1536 numbers is a point in a 1536-dimensional space. The word "dimension" just means "how many numbers per text".

2. The axes are learned, not named

Nobody decides that dimension 17 means "spiciness". The model discovers useful axes during training, and each one is usually a blend of many ideas.

You cannot label them, but the *distances* between points still mean something.

An embedding is a point in a very high-dimensional space, and nearby points mean similar things.

1536 dimensions simply means 1536 numbers per text. That is all the word means.

Worked example: similarity with three numbers

Before trusting a real model, do the whole thing by hand with vectors small enough to read. We score five dishes on three axes, `[spicy, sweet, crunchy]`, and then ask which dish best matches the query "I want something sweet".

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

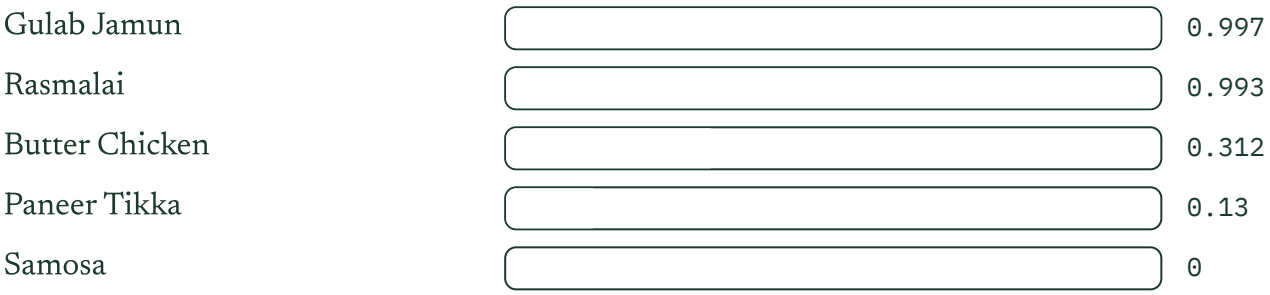
#                     spicy  sweet  crunchy
dishes = {
    "Gulab Jamun":    [0.05,  0.95,  0.05],
    "Rasmalai":       [0.10,  0.85,  0.00],
    "Butter Chicken": [0.60,  0.20,  0.10],
    "Paneer Tikka":   [0.70,  0.10,  0.30],
    "Samosa":         [0.50,  0.00,  0.90],
}
query = np.array([[0.0, 1.0, 0.0]])          # "something sweet": all weight

names = list(dishes)
matrix = np.array([dishes[n] for n in names])

scores = cosine_similarity(query, matrix)[0]
```

```
for name, score in sorted(zip(names, scores), key=lambda x: -x[1]):
    print(f"{score:.3f}  {name}")
```

```
0.997  Gulab Jamun
0.993  Rasmalai
0.312  Butter Chicken
0.130  Paneer Tikka
0.000  Samosa
```



cosine similarity to the query. The two sweet desserts win.

Look at what happened. We never wrote the word "dessert" anywhere. The ranking came purely from **geometry**: the query points mostly along the sweet axis, so the dishes pointing the same way score highest. A real embedding model does the same thing, only with 1536 learned axes instead of 3 hand-made ones, and it learns the scores from reading enormous amounts of text.

tip

Try changing the query to `[0.8, 0.0, 0.7]` ("spicy and crunchy"). The Samosa should jump to the top. Moving a point in the space changes what it is close to, and that is all search ever does.

Inside an embedding model

So where do the numbers come from? You can treat a model as a black box, but seeing the steps once removes the mystery. We follow one sentence, *"Butter chicken is spicy"*, through a

small embedding model that outputs 384 numbers. Larger models use the same recipe with bigger numbers. There are six steps.

1. Tokenize

Models do not read letters or whole words. A **tokenizer** chops the text into pieces from a fixed vocabulary of about 30,000 entries. Rare words are split into smaller known pieces, which is how a model copes with words it has never seen. Special start and end markers are added around the text, so our sentence becomes six pieces.

2. Convert tokens to IDs

Every piece in the vocabulary has an integer ID, so the six pieces become six whole numbers. Nothing about meaning exists yet. The IDs are just positions in the vocabulary.

3. Look up a vector for every token

Inside the model sits a big learned table, the **embedding matrix**, with one row per vocabulary entry and one column per dimension. Each ID selects its row, so our six tokens become a grid of 6 rows by 384 numbers. At this stage "bank" has the same vector in every sentence.

4. Transformer layers mix in context

A stack of transformer layers lets every token look at all the others through **attention**, and rewrites each token's vector using its neighbours. Compare "I deposited cash at the *bank*" with "We sat on the river *bank*". The token "bank" starts with identical numbers in both, but after the layers its vector differs, because the surrounding words pull it towards money in one sentence and towards water in the other. The grid keeps its shape, 6 x 384, but the numbers now carry meaning in context.

5. Pool into one vector

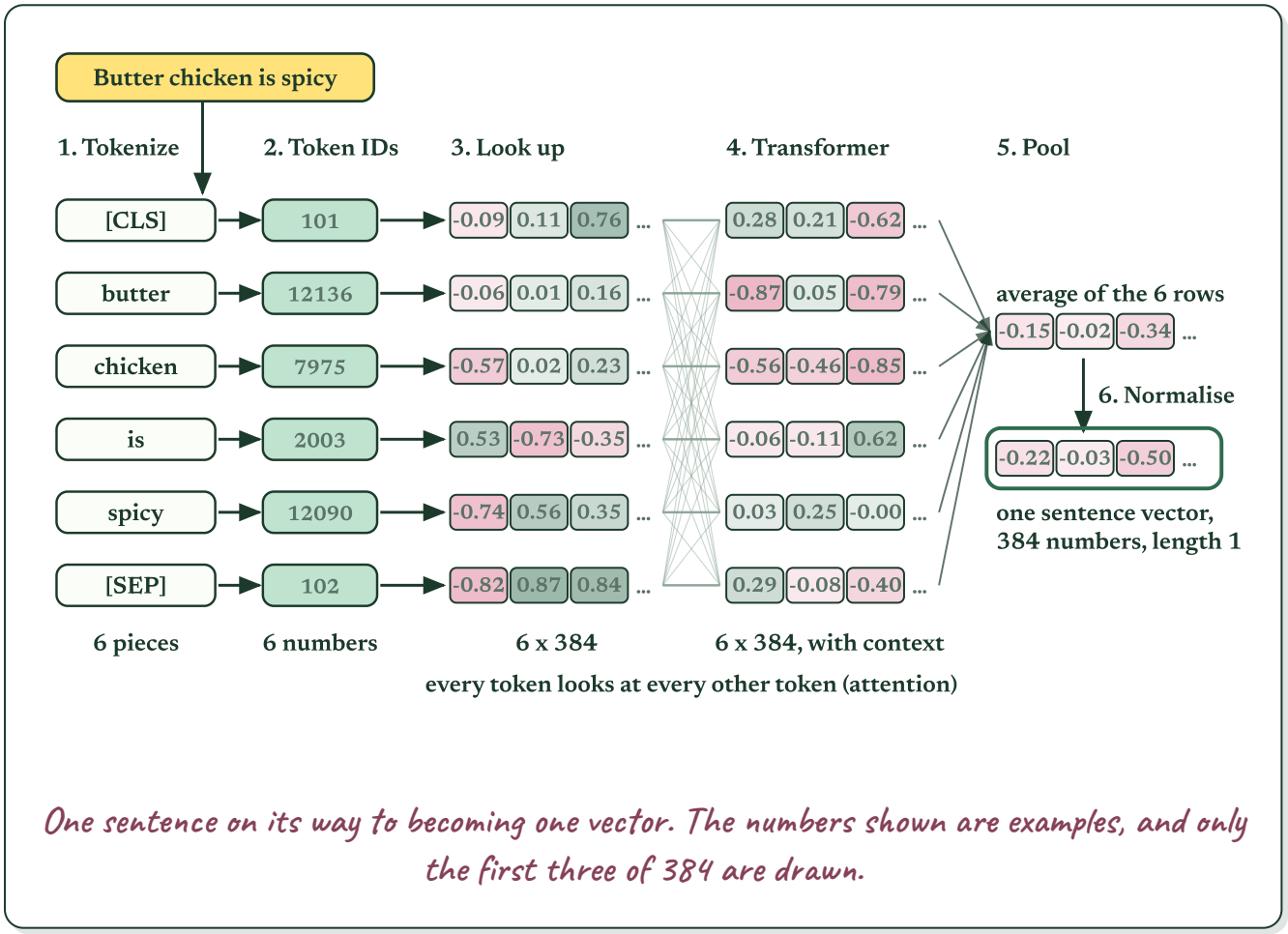
We need one vector for the whole sentence, not six. **Pooling** combines the six token vectors into one, and the common choice is simply their average.

6. Normalise

Many models finally scale the result so its length is exactly 1. The output is one list of 384 numbers that represents the whole sentence, and this is the embedding you store and search.

The whole journey in one picture

Read the picture from left to right. The sentence is cut into tokens, each token turns into an ID and then into its own row of numbers, the transformer mixes information between the rows, pooling squeezes six rows into one, and normalising tidies that one row into the final sentence vector.



One sentence on its way to becoming one vector. The numbers shown are examples, and only the first three of 384 are drawn.

Where did the numbers come from? Training

The model's numbers were learned by showing it millions of pairs of texts. Pairs that belong together, such as a question and its answer, are pulled closer in the space. Pairs that do not belong together are pushed apart. After many rounds, nearby points mean similar things. That is the only reason "automobile" lands near "car".

Training Pair that matches → Pull the vectors closer

Training Pair that does not match → Push the vectors apart

note

You rarely train an embedding model yourself. Treat it as a function: text in, one fixed-length vector out. But now you know the vector is built from tokens, context and pooling, not from a keyword list.

Go deeper The Illustrated Transformer Sentence Transformers: usage

Dimensions in practice

With the meaning clear, here are the practical facts you will meet every day. First, the **shape** of what a model returns. One text gives one row of numbers, and the length of that row is the dimension.

```
import numpy as np

one = np.array(embeddings[0])
batch = np.array(embeddings)

print(one.shape)      # (1536,)      one text  -> one vector with 1536 numbe
print(batch.shape)    # (2, 1536)    two texts -> two rows of 1536 numbers e
```

Copy

3 dimensions [0.8 0.1 0.4]

1536 dimensions [0.21 -0.13 0.88 -0.05 0.37 -0.62 0.14 0.71 …]

Common models and their dimensions

Model	Dimensions	Notes
`all-MiniLM-L6-v2`	384	Small, free, runs on a laptop
`all-mpnet-base-v2`	768	Larger open model, stronger quality
`text-embedding-3-small`	1536	OpenAI, hosted API, default size
`text-embedding-3-large`	3072	OpenAI, highest quality of the family

What dimensions cost

Every number is stored as a 32-bit float, which is 4 bytes. So one vector costs `dimensions x 4` bytes, and the bill grows with both the dimension and the number of chunks. The same factor also slows every search and every network transfer.



memory for 1 million vectors, vectors only. Real systems also store the text, metadata and the index.

Shrinking the dimension on purpose

OpenAI's third-generation embedding models are trained so that a shortened vector still keeps most of its usefulness. You can ask the API for fewer numbers directly, trading a little quality for a large saving in storage and speed:

```
response = client.embeddings.create(
    model="text-embedding-3-large",
    input="Butter chicken is spicy",
    dimensions=256,             # ask for 256 numbers instead of 3072
)
```

```
print(len(response.data[0].embedding))      # 256
```

Fewer dimensions

Smaller index, faster search, cheaper storage and transfer, but a little less detail captured.

More dimensions

Room for finer distinctions, but more memory, slower search and a bigger bill.

note

Common question: does more dimensions mean a better RAG system? Not by itself. Extra dimensions give the model more room to tell similar chunks apart, which can lift recall and precision a little, mostly in large collections full of near-duplicate text. But you pay for every dimension, on every chunk, in memory, storage, search time and cost.

Choice	You gain	You lose
Fewer (for example 256)	Much less memory, faster search, lower cost	A little recall and precision, as fine distinctions blur
More (for example 3072)	Finer distinctions, a small gain in recall and precision	Around 12 times the memory of 256, slower search, a bigger bill

Chunking, the quality of the embedding model and metadata filters usually matter far more than the number of dimensions. A sensible approach is to start with the model's default, measure recall on your own questions, and shorten the vectors only if cost or speed forces you to.

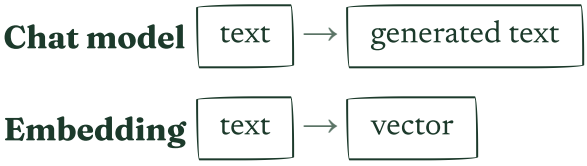
watch out

Three myths to avoid. A single dimension is **not** a human feature such as "Java" or "AI", because meaning is spread across all of them. More dimensions does **not** automatically

mean a better model. And every vector in one collection must have the **same** number of dimensions.

Generating embeddings with the OpenAI SDK

Chat models turn text into text. Embedding models turn text into vectors. They use different endpoints.




```
pip install openai numpy scikit-learn qdrant-client
```

Copy

```python
from openai import OpenAI

client = OpenAI()   # reads OPENAI_API_KEY from your environment

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=[
        "Java is a programming language",
        "Python is used for machine learning",
    ],
)

embeddings = [item.embedding for item in response.data]

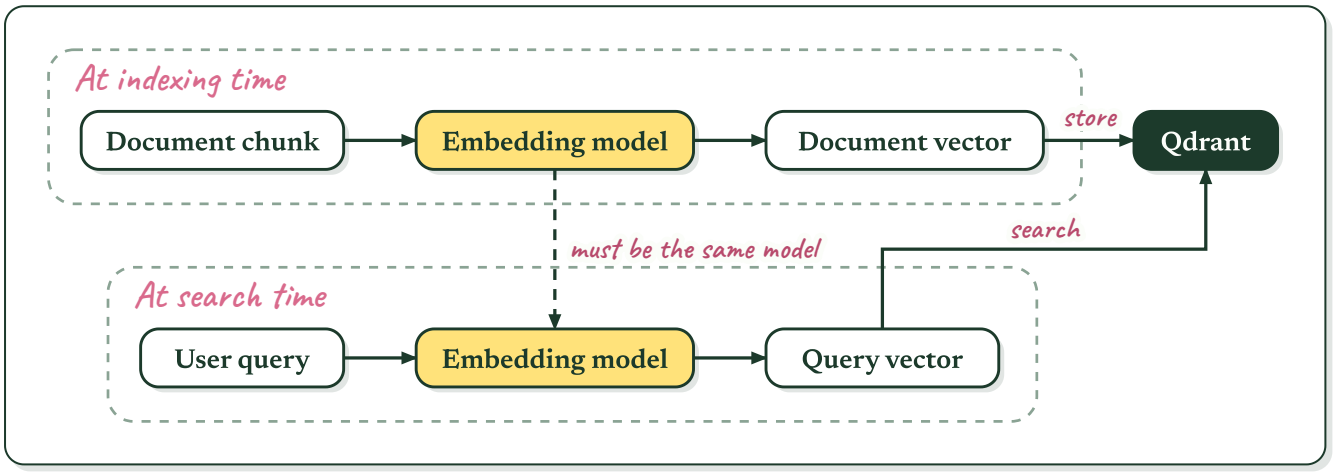
print(len(embeddings))        # 2 texts
print(len(embeddings[0]))     # 1536 numbers each
print(embeddings[0][:5])      # first five numbers
```

Copy

Model names and limits change, so always confirm against the official embeddings guide.

Document vs query embeddings

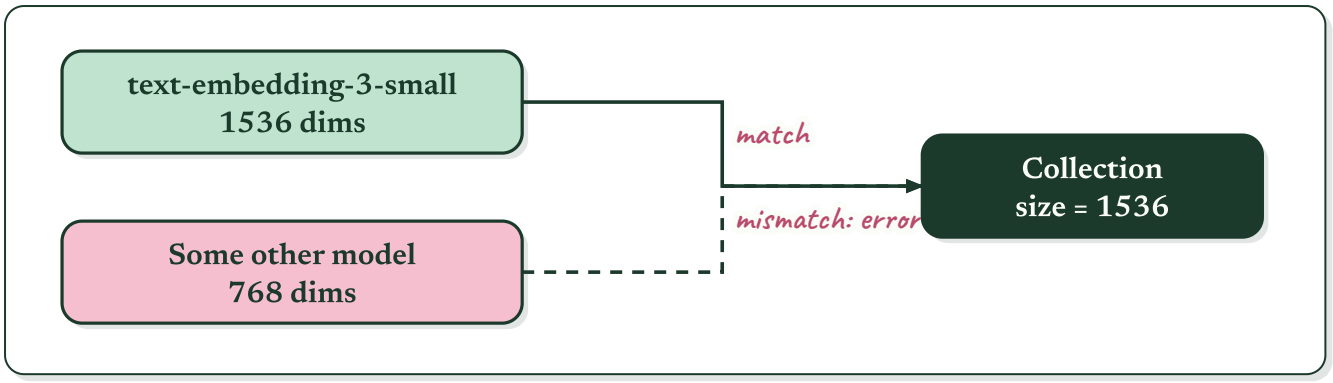
Both sides of RAG use an embedding model, at different moments:



The query and the stored documents must live in a **compatible embedding space**. A vector from model A is meaningless next to a vector from model B, even if both have the same length.

Embedding dimensions and compatibility

Suppose your model outputs 1536 dimensions. The vector database collection must be configured for exactly 1536, or inserts will fail. Qdrant sets dimensionality and metric at the collection level.




```
dim = len(embeddings[0])    # always read it from the model, never hard code
print("Collection must use size =", dim)
```

Copy

watch out

If you ever change the embedding model, you must re-embed and re-index *every* document. Mixing models inside one collection quietly destroys retrieval quality.

part four

Measuring similarity

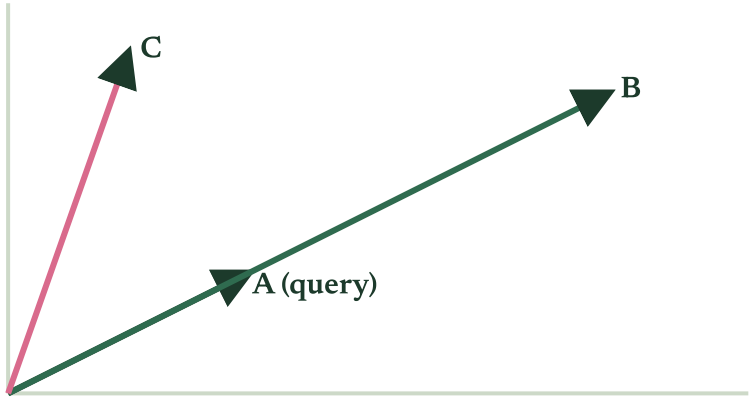
Distance metrics at a glance

Once text is a vector, "how similar are these two texts?" becomes "how close are these two vectors?" There are three common ways to measure that.

Metric	Intuition	Retrieval direction
Cosine	Angle between vectors	higher = more similar
Euclidean / L2	Straight-line distance	lower = closer
Dot product	Alignment plus magnitude	higher = better

Metric	Intuition	Retrieval direction
Manhattan / L1	City-block distance: sum of per-dimension gaps	lower = closer

The right metric depends on how the embedding model was trained. Qdrant supports Cosine, Dot, Euclidean and Manhattan.



A and B point the same way: cosine says "very similar". C points elsewhere: not similar.

Notice B is much longer than A. Cosine ignores length and only looks at direction. L2 would call A and B *far apart*. This is why the metric must match the model.

Cosine similarity

```
              A · B
cos(A,B) = -------------
           ||A|| * ||B||

Small angle between arrows  ->  high similarity
```

Values run from -1 to 1. For text embeddings you will mostly see values between about 0 and 1. With scikit-learn it is one call, see the cosine_similarity docs:

```
from sklearn.metrics.pairwise import cosine_similarity

scores = cosine_similarity(query_embedding, document_embeddings)
```

Copy

Euclidean / L2 distance

The ordinary straight-line distance between two points, extended to many dimensions.

```
from sklearn.metrics.pairwise import euclidean_distances

distances = euclidean_distances(query_embedding, document_embeddings)
```

Copy

> *Cosine → higher score = more similar.*
> *L2 → lower distance = more similar.*
>
> Reading the number in the wrong direction is the most common beginner bug in this topic.

Reference: scikit-learn euclidean_distances.

Dot product / inner product

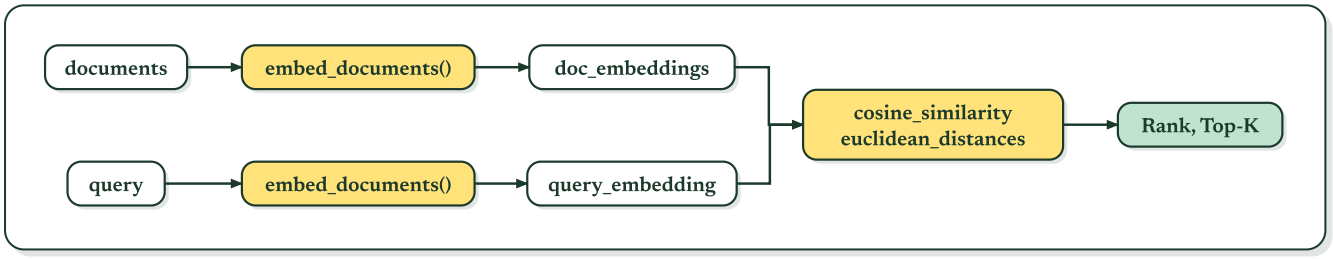
```
A · B = a1*b1 + a2*b2 + a3*b3 + ...
```

It measures how aligned two vectors are, and it is also affected by their length (magnitude). If all vectors are normalised to length 1, dot product and cosine give identical rankings. Many embedding models return normalised vectors, which is why you will see both used.

Understand the intuition first. You do not need to memorise the mathematics.

Practical lab: generate and compare embeddings

Time to see it work. We embed four documents and one query, then ask both metrics which document is closest.



lab1_compare.py

```
from openai import OpenAI
from sklearn.metrics.pairwise import cosine_similarity, euclidean_distances

client = OpenAI()
MODEL = "text-embedding-3-small"

def embed_documents(texts):
    resp = client.embeddings.create(model=MODEL, input=texts)
    return [d.embedding for d in resp.data]

documents = [
    "Java is a popular programming language used for backend development.",
    "Python is widely used for machine learning and artificial intelligence.
    "Delhi is the capital of India.",
    "Spring Boot is a Java framework for backend applications.",
]
query = "Which programming language is commonly used for AI applications?"
```

```
doc_embeddings = embed_documents(documents)
query_embedding = embed_documents([query])      # a list holding one vector

cos = cosine_similarity(query_embedding, doc_embeddings)[0]
l2 = euclidean_distances(query_embedding, doc_embeddings)[0]

for doc, c, d in zip(documents, cos, l2):
    print(f"cos={c:.3f}  l2={d:.3f}  {doc}")
```

Illustrative output (your exact numbers will differ):

```
cos=0.452  l2=1.047  Java is a popular programming language ...
cos=0.571  l2=0.926  Python is widely used for machine learning ...   <- bes
cos=0.103  l2=1.340  Delhi is the capital of India.
cos=0.379  l2=1.115  Spring Boot is a Java framework ...
```



cosine scores from the output above. With top_k=2, the green rows are returned.

Two things to notice. The Python sentence wins even though the query never says "Python": that is semantic search. And the cosine winner is also the L2 winner, but one is the *highest* number and the other the *lowest*.

Manual semantic retrieval

You now have every ingredient of the core retrieval algorithm:

1. Embed the query
2. Compare it with every document vector

3. Calculate similarity
4. Rank the results
5. Return the Top-K




```python
import numpy as np

def retrieve(query_embedding, document_embeddings, documents, top_k=2):
    scores = cosine_similarity(query_embedding, document_embeddings)[0]
    ranked = np.argsort(scores)[::-1]          # highest score first
    return [{"document": documents[i], "score": float(scores[i])}
            for i in ranked[:top_k]]

for hit in retrieve(query_embedding, doc_embeddings, documents):
    print(round(hit["score"], 3), hit["document"])
```

Copy

> *This small function is the bridge between vector maths and a vector database.*
>
> Everything Qdrant does is this function, made fast, filterable and persistent.

Which metric should I use?

All four metrics turn two vectors into one number, but they measure **different geometric things**. Picking the right one depends on how your vectors were produced and what "similar" means in your domain. Start with this map, then we will look at real examples for each choice.

Metric	Measures	Ignores	Best for
Cosine	Angle, meaning direction	Vector length	Text search, document embeddings, RAG
Dot product	Angle and length together	Nothing	Recommendations, already-normalised embeddings
Euclidean (L2)	Straight-line distance	Nothing	Images, coordinates, physical measurements
Manhattan (L1)	Sum of per-dimension gaps	Nothing, but resists one huge outlier	Noisy, high-dimensional or grid-like data

Cosine vs Euclidean: does length matter?

Cosine asks *"do these vectors point the same way?"* Euclidean asks *"how far apart are their endpoints?"* Choose by deciding whether the length of a vector means anything.

Pick cosine: document search

A 200-word summary of AI and a 10,000-word research paper on AI cover the same topic. With raw word counts, the long paper has huge numbers, so Euclidean puts it **far away** from the summary. Cosine ignores length and correctly calls them near-identical.

Pick Euclidean: pixels and positions

A bright red pixel `(255, 0, 0)` and a dark red pixel `(10, 0, 0)` point the same way, so cosine says 1.0. But for image retrieval, brightness is a real difference. Euclidean sees the gap, cosine cannot.

Dot product vs cosine: does popularity matter?

Dot product equals cosine multiplied by both lengths: `A · B = ‖A‖ ‖B‖ cos(θ)`. Use it when a bigger vector should win, such as a blockbuster outranking an indie film that matches the same taste.

`metrics_compare.py`

```
import numpy as np
from sklearn.metrics.pairwise import (
    cosine_similarity, euclidean_distances, manhattan_distances,
)

user       = np.array([[0.8, 0.2, 0.0]])    # likes action, a little comedy
indie      = np.array([[0.8, 0.2, 0.0]])    # niche action film
blockbuster = np.array([[8.0, 2.0, 0.0]])    # same taste, 10x the popularit

for name, movie in [("indie", indie), ("blockbuster", blockbuster)]:
    print(name,
          "cos =", round(cosine_similarity(user, movie)[0][0], 2),
          "dot =", round((user @ movie.T)[0][0], 2),
          "l2 =", round(euclidean_distances(user, movie)[0][0], 2),
          "l1 =", round(manhattan_distances(user, movie)[0][0], 2))
```

Copy

```
indie       cos = 1.0   dot = 0.68   l2 = 0.0    l1 = 0.0
blockbuster  cos = 1.0   dot = 6.8    l2 = 7.42   l1 = 9.0
```

Cosine: indie		1
Cosine: blockbuster		1
Dot product: indie		0.68
Dot product: blockbuster		6.8

Cosine cannot tell the two films apart. Dot product ranks the blockbuster 10x higher. Euclidean calls the blockbuster far away, which is the wrong signal here.

tip

If your embeddings are already normalised to length 1, cosine and dot product give the **same ranking**. OpenAI's embeddings are normalised (see the embeddings guide), so for RAG they are interchangeable. Qdrant also normalises vectors on insert for the Cosine metric, then uses the cheaper dot product internally.

Manhattan vs Euclidean: do outliers matter?

Euclidean squares each gap before adding them up, so one huge difference dominates the result. Manhattan adds the gaps as they are.

Grid movement

A delivery rider cannot cut through buildings. The real trip is blocks east plus blocks north, which is exactly Manhattan distance, not the straight line a crow would fly.

Noisy measurements

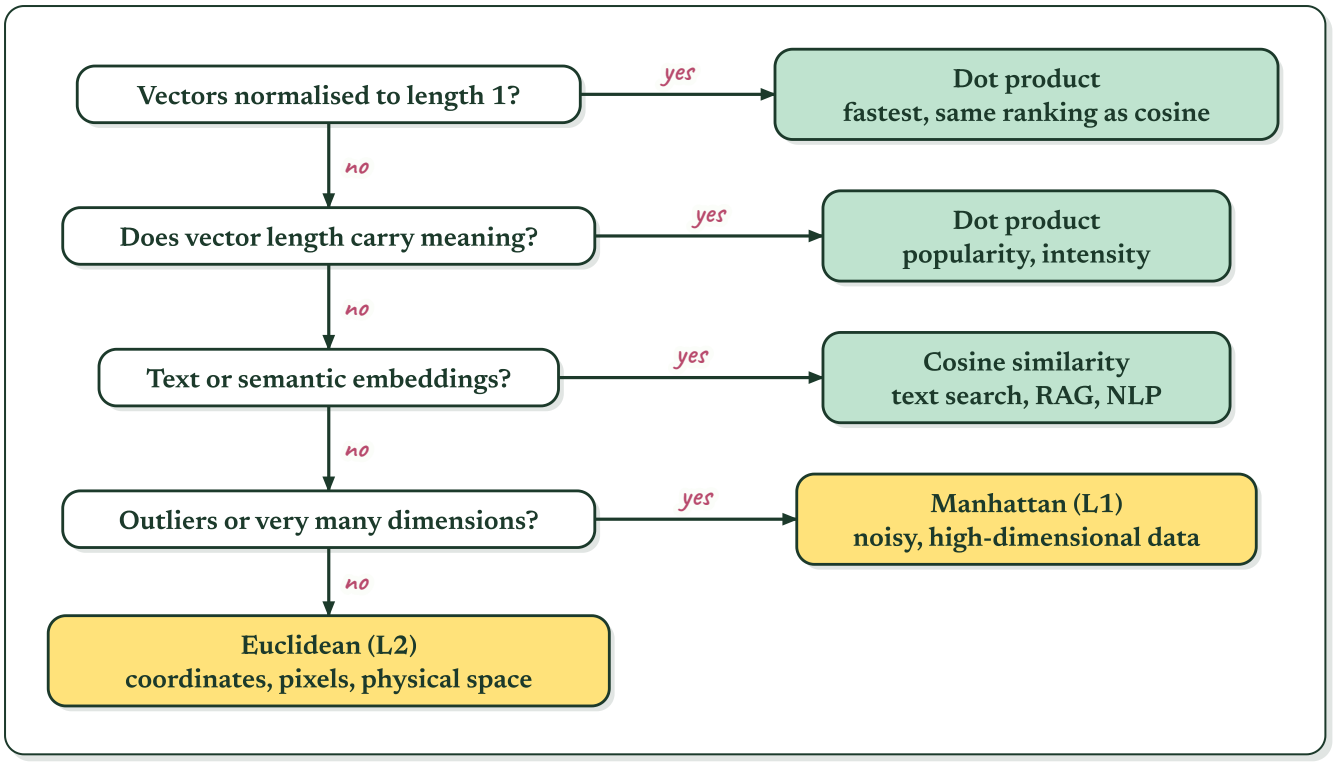
A transaction vector with 100 normal features and one corrupted value. Euclidean squares that glitch and lets it drown the other 100 signals. Manhattan stays robust.

Very high dimensions

With hundreds of dimensions, Euclidean distances tend to bunch together and lose contrast. Manhattan often keeps a clearer gap between near and far.

A decision guide with real cases

When you are stuck, walk down this tree. For most RAG work you will land on cosine or dot product within the first two questions.



Real-world cheat sheet

Scenario	Choose	Why
Chatbot over company documents (your RAG project)	Cosine	Meaning lives in direction, and a long policy PDF should not be punished for its length.
"Customers also liked" on a shopping site	Dot product	A best-seller should outrank an obscure item with the same taste profile.
Finding visually similar product photos	Euclidean	Absolute intensity and position matter, not just proportions.
Estimating delivery distance on a city grid	Manhattan	Movement follows streets, so you sum blocks, not diagonals.
Fraud scoring on sensor-like features with glitches	Manhattan	One corrupted feature should not dominate the score.

Always use the metric your embedding model was trained for.

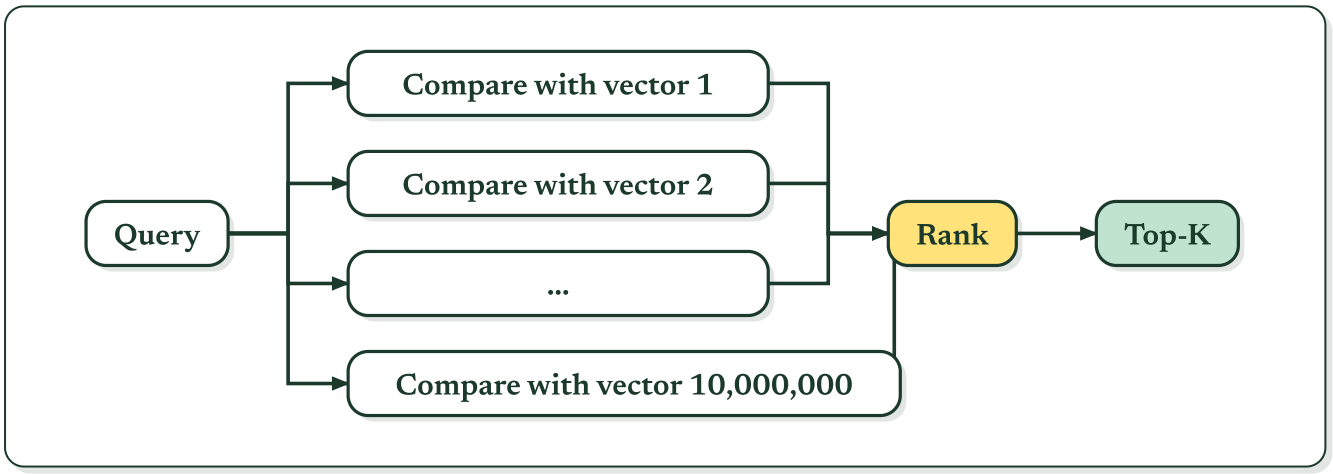
The model's documentation tells you. A great metric on the wrong model still gives poor retrieval, and the collection's distance setting in Qdrant must match.

part five

Vector search with Qdrant

Why do we need a vector database?

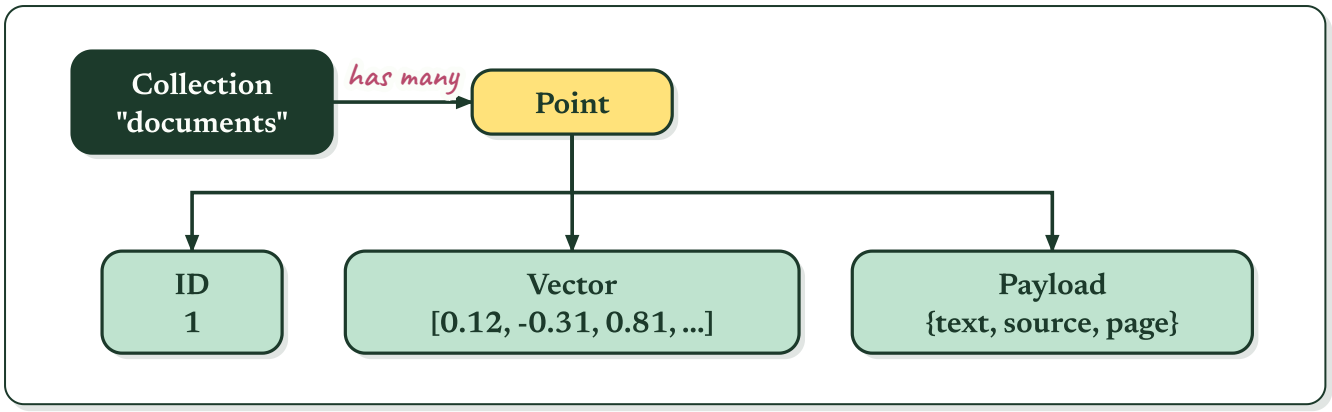
With 10 documents, comparing against all of them is instant. With 10 million, it is not:



A vector database gives you optimised storage, indexing, search, filtering and operational features such as persistence and scaling. Here we use Qdrant. For a plain-language intro, read What is a vector database?

The Qdrant data model

Three words are enough to describe all of Qdrant's data:



Collection

A named, searchable set of points. Roughly a table.

Point

One record: an ID, a vector and an optional payload. Roughly a row.

Payload

JSON-like metadata attached to a point. This is where your chunk text and filters live.

> *Go deeper* Collections Points Payload

Local, in-memory mode

You can learn the entire Qdrant API without installing anything:

```
from qdrant_client import QdrantClient, models

client = QdrantClient(":memory:")      # lives in RAM, vanishes when the scri
```

Copy

Later the same code can point to a Docker server or a managed cloud cluster by changing one line (`QdrantClient(url="http://localhost:6333")`). See the Qdrant quickstart

for the Docker route.



Create a collection

```
client.create_collection(
    collection_name="documents",
    vectors_config=models.VectorParams(
        size=1536,                          # must match the embedding model
        distance=models.Distance.COSINE,    # must match how the model was tra
    ),
)
```

Only two decisions matter here: the **vector dimension** and the **distance metric**. The dimension should come from the real embedding you generated, as we saw earlier.

Upsert points

"Upsert" means insert, or update if the ID already exists. Conceptually: embedding becomes vector plus metadata, which goes into Qdrant.

```
points = [
    models.PointStruct(
        id=i,
        vector=vec,
        payload={"text": doc, "category": "demo"},
    )
    for i, (doc, vec) in enumerate(zip(documents, doc_embeddings))
]
```

```
client.upsert(collection_name="documents", points=points)
```

Chunk text → Embedding → Vector + payload → Point in Qdrant

Notice that we reuse the `documents` and `doc_embeddings` from the lab. Same vectors, new home.

Search Qdrant

Copy

```
results = client.query_points(
    collection_name="documents",
    query=query_embedding[0],     # a single vector, not a list of vectors
    limit=3,
    with_payload=True,
).points

for p in results:
    print(round(p.score, 3), p.payload["text"])
```

Each result carries a point ID, a similarity score and the payload. Compare the ranking with your manual `retrieve()` output: it should be the same. See the search documentation.

Manual

Query vector → Compare with all → Sort → Top-K

Qdrant

Query vector → Vector index → Search → Ranking → Top-K

The fundamental problem has not changed. Qdrant wraps an optimised search system around it.

part six

Searching strategies

KNN and Top-K

KNN stands for K-Nearest Neighbours: find the K vectors closest to the query. If K = 5 you want the five nearest. In RAG, those become the context for the LLM.



Changing K shifts several things at once:

Recall
higher with bigger K

Noise
also higher

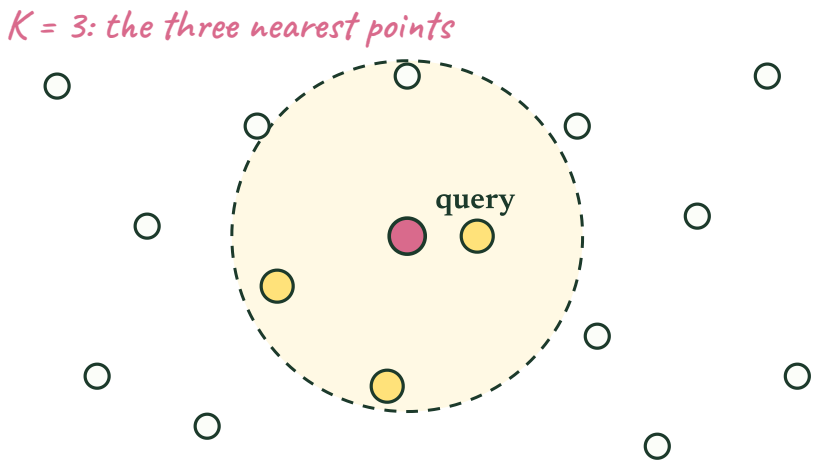
Prompt size
grows

Cost
grows

Latency
grows

note

What is recall? Recall is the share of the *right* answers that your search actually returned. Say 4 chunks in your documents truly answer a question, and the top 10 results contain 3 of them. Recall is 3 out of 4, or 75%, and the missing chunk is invisible to the LLM. Its partner measure, **precision**, asks the opposite: of the 10 results, how many were useful? Here that is 3 out of 10. A bigger K raises recall and lowers precision. The same measure is used to judge approximate indexes, by how many of the true nearest neighbours they manage to find.



KNN with K = 3: take the three points closest to the query.

There is no universally correct K. More context is not automatically better context: the research paper Lost in the Middle found that models use information in the middle of a long context less reliably.

Exact (flat) search

The simplest search compares the query against every vector and returns the exact ranking.

Good for

- Small datasets
- Baselines
- Evaluation
- Understanding retrieval

Limitation

- Cost grows with every vector you add

```
results = client.query_points(
    collection_name="documents",
    query=query_embedding[0],
    limit=3,
```

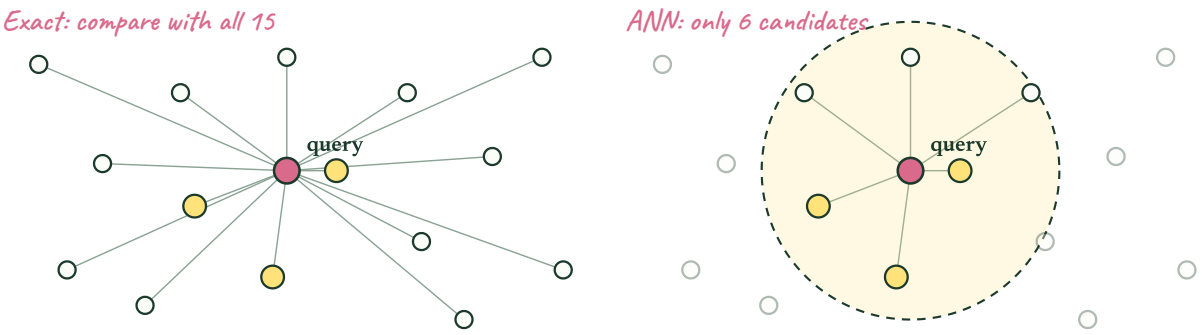
```
    search_params=models.SearchParams(exact=True),    # brute force, exact ra
)
```

Exact search is also how you build a **ground-truth baseline** to check how good the approximate search is. Always confirm the call shape against your installed client version.

Approximate nearest neighbour (ANN)

ANN avoids comparing everything. It uses a prebuilt index to jump straight to promising candidates, accepting that it may occasionally miss a true neighbour. There are several families of ANN index. Graph-based **HNSW**, which Qdrant uses, is the one we study closely. Cluster-based **IVF** is the other classic, and a short comparison follows after HNSW.

Query → ANN index → Promising candidates → Top-K



Same query, same three answers. ANN did less than half the work. At 10 million vectors the gap is enormous.

More search effort ↔ better recall ↔ more latency and compute.

Every ANN setting is a dial on this one trade-off.

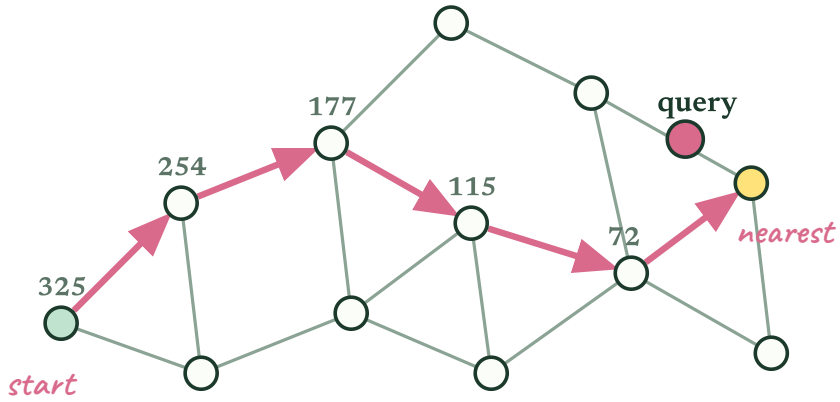
HNSW: the idea

HNSW is the index Qdrant uses to make vector search fast. The easiest way to understand it is to build it up in small steps, each solving a problem left by the one before. The name **Hierarchical Navigable Small World** is just three ideas stuck together:

Word in the name	Plain meaning
Navigable	The vectors are linked into a graph that you can walk along, always moving closer to what you want
Small World	A few long-range links mean any vector is only a few hops from any other
Hierarchical	The graph is stacked in layers, from a high-level overview at the top to full detail at the bottom

Step 1: link every vector to its close neighbours

Imagine every vector is a person, and each person knows a handful of the people who are most similar to them, their **friends**. Join those friendships up and you have a graph. To search, pick any starting person and ask: *which of my friends is closest to what the query is looking for?* Move to that friend, and ask again. Stop when none of your friends is closer than you are.



Greedy search: at every stop, hop to the friend closest to the query. The small numbers are each stop's distance to the query, and they shrink all the way.

This is called **greedy search**, and notice what it never does: it never looks at the vast majority of the vectors. It only examines the few friends of each stop on its way, and the distances shrink at every hop. With a million vectors, that can be a few hundred distance checks instead of a million.

The problem with one flat graph

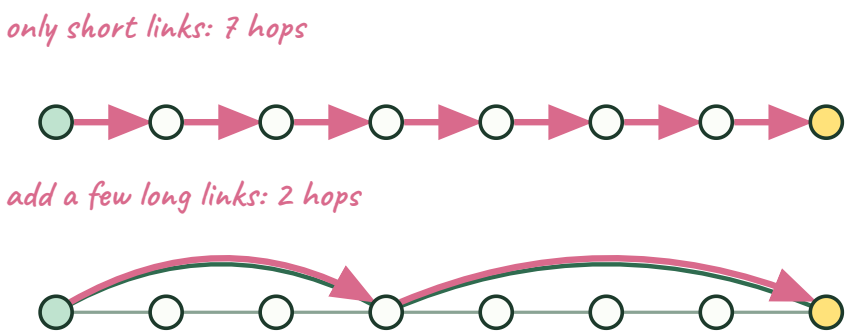
- If you start far from the answer and everyone only knows nearby people, you need **many small hops** to get across the map.
- You can also get **stuck**: a vector whose friends are all farther away than itself looks like a winner even when a better one exists elsewhere. This is one reason HNSW is only *approximate*.

The next two ideas fix exactly these two problems.

HNSW: shortcuts and layers

Step 2: add a few long-range shortcuts (the small world)

Think of how few introductions connect any two people on Earth. It works because a few people know someone far away. If we give some vectors a link to a distant vector, the long walk collapses into a short one:

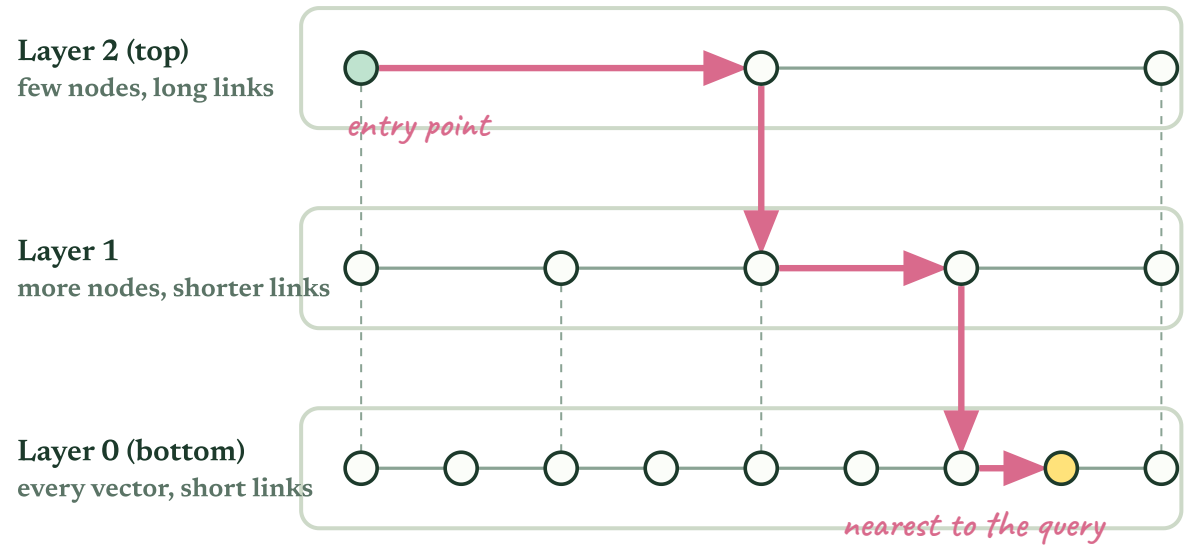


A few long-range links turn a long walk into a short one. This is the "small world" idea, the same reason any two people are only a handful of introductions apart.

Step 3: stack the graph in layers (the hierarchy)

Instead of mixing short and long links in one graph, HNSW separates them into **layers**. Every vector lives in the bottom layer. Each vector is also given, by something like a coin toss, a chance of appearing in the layer above, and a smaller chance in the one above that. So the higher the layer, the fewer vectors it holds, and the farther apart its neighbours are.

It is the same idea as travelling across a country: take a **flight** between hub airports, then a **motorway**, then a **main road**, then walk the last few streets. Each level of transport covers a lot of ground and hands you over to a finer one.



The same vectors in three layers. Dashed lines show that a vector in a higher layer is the very same vector as the one below it. The search enters at the top and drops down whenever it cannot get closer.

> *Top layers give you long jumps to the right region. The bottom layer gives you the fine detail to find the exact neighbour.*
>
> That is the whole trick. Coarse to fine, like zooming into a map.

HNSW: a search, step by step

Here is exactly what happens when a query arrives. Follow the pink path in the layer figure while you read.

1. Start at the fixed **entry point** in the top layer.
2. Walk greedily along that layer: keep moving to whichever friend is closest to the query. Stop when no friend is closer.
3. **Drop down** one layer, starting from the node where you stopped.

4. Repeat the greedy walk, then drop again, until you reach the bottom layer.
5. In the bottom layer, do a wider search. Keep a list of the `ef` best candidates seen so far, keep exploring their friends, and stop when no candidate can be improved.
6. Return the best `K` from that list.

A walk-through with made-up distances (smaller means closer to the query):

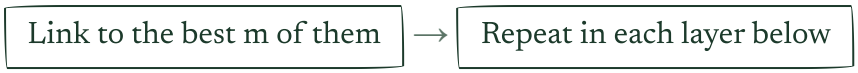
Step	Layer	Where we are	What happens
1	Top	Entry point A, distance 0.92	A's friend E is closer (0.61), so move to E
2	Top	E, distance 0.61	No friend is closer, so drop down
3	Middle	E, distance 0.61	Friend G is closer (0.38), so move to G
4	Middle	G, distance 0.38	No friend is closer, so drop down
5	Bottom	G, distance 0.38	Explore G's friends and keep the `ef` best candidates, such as H (0.12) and J (0.19)
6	Bottom	Candidates H, J, ...	Nothing better appears, so return the top K

In six steps we examined a handful of vectors out of millions, and the answer is almost always the true nearest neighbour. "Almost always" is the price of being approximate. That fraction of true neighbours found is what the **recall** measure from earlier counts.

HNSW: building the graph

The graph is not drawn by hand. It is built one vector at a time, and the build uses the very same search you just saw. When a new vector arrives:

New vector → Pick its top layer at random → Search for its nearest neighbours →



- The random draw gives most vectors a top layer of 0, a few a top layer of 1, even fewer a top layer of 2, and so on. That is what produces the pyramid shape.
- In every layer the vector belongs to, it searches for its closest existing neighbours, then connects to the best `m` of them.
- If a neighbour now has too many links, the weakest ones are dropped, so the graph never grows unbounded.

Because each insertion performs a search, building the index takes noticeably longer than simply storing vectors, and the links take extra memory on top of the vectors themselves. In return, every later query is fast. This is why HNSW is a trade: you spend build time and memory to buy query speed.

> *Go deeper* Pinecone: HNSW explained · Original HNSW paper · Qdrant indexing docs

HNSW tuning: m, ef_construct, ef

Three knobs control the trade between speed, memory and recall. In plain words, using the friends analogy:

`m` : friends per node
How many neighbours each vector links to. More friends make the graph better connected and easier to navigate, but every extra link costs memory.

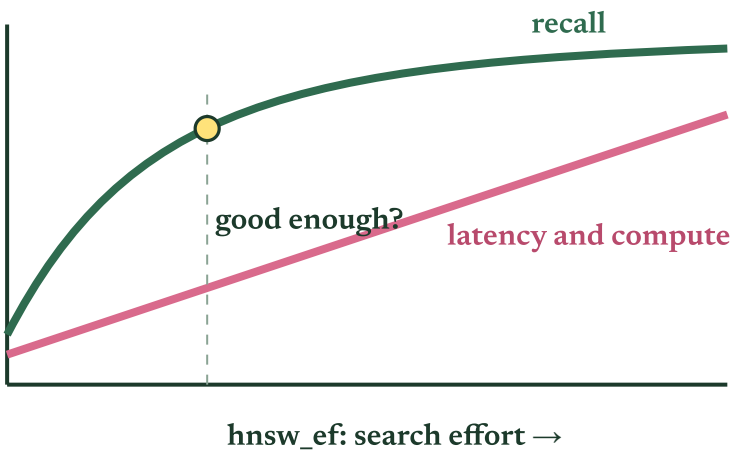
`ef_construct` : care when joining
How many candidates the builder considers when choosing a new vector's friends. More care builds a better graph, but indexing is slower.

`hnsw_ef` : leads to follow

How many candidates the search keeps alive in the bottom layer. More leads mean fewer missed neighbours, but each query takes longer.

Know the *purpose* of each knob. Exact defaults change between versions. Qdrant's defaults, `m=16` and `ef_construct=100`, are a sensible starting point.

Parameter	When it applies	Purpose
`m`	Index build	Maximum graph connections per node. Higher: better recall, more memory.
`ef_construct`	Index build	Search effort while building the graph. Higher: better graph, slower indexing.
`hnsw_ef`	Query time	Search effort per query. Higher: better recall, more latency.



Recall flattens out while cost keeps climbing. Tune ef until recall is good enough, not maximal.

```
# build-time settings, chosen when the collection is created
client.create_collection(
    collection_name="tuned",
```

```
    vectors_config=models.VectorParams(size=1536, distance=models.Distance.C
    hnsw_config=models.HnswConfigDiff(m=16, ef_construct=100),
)

# query-time setting, changeable on every request
client.query_points(
    collection_name="tuned",
    query=query_embedding[0],
    limit=5,
    search_params=models.SearchParams(hnsw_ef=128),
)
```

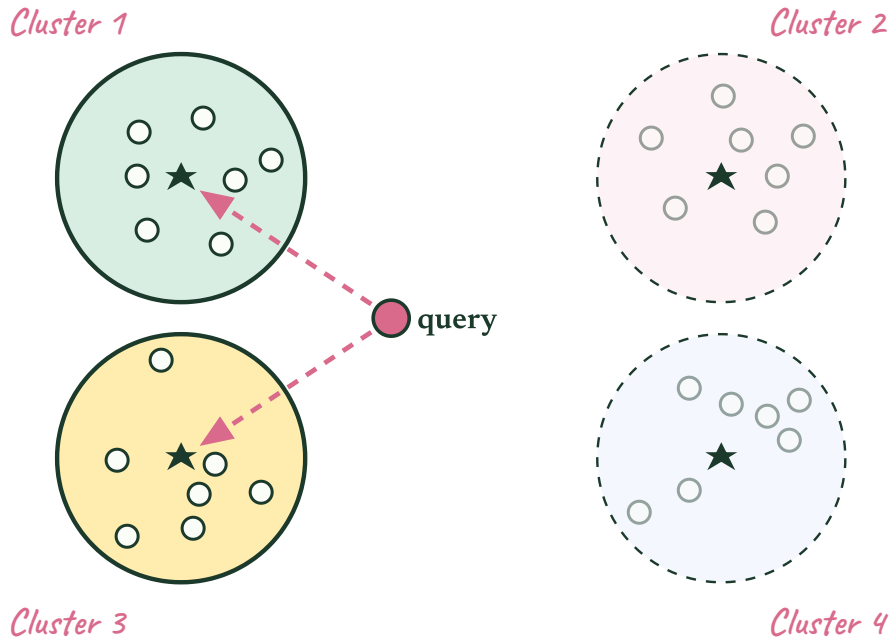
Higher search effort → Potentially higher recall → More computation and latency

IVF compared with HNSW

HNSW is not the only way to do approximate search. The other classic family is **IVF**, short for *Inverted File index*. You will meet it in many vector tools, for example the `IVFFlat` index in PostgreSQL's pgvector extension.

How IVF works

1. **Group ahead of time.** A clustering step sorts all the vectors into a chosen number of groups, called *lists* or clusters. Each cluster has a centre.
2. **Narrow down at query time.** Compare the query only with the cluster centres and pick the few nearest ones. The number you pick is often called *nprobe* (or *probes*).
3. **Search inside those clusters only.** The vectors in the other clusters are never touched.



IVF in one picture. Vectors are grouped into clusters ahead of time, and the stars are the cluster centres. At query time only the nearest few clusters (here 2 of 4) are searched. The faded clusters are skipped.

It is the library idea: instead of reading every book, you walk to the right shelves first. The trade-off is also clear from the picture. If the true nearest neighbour happens to sit in a cluster you did not probe, you miss it. Probing more clusters raises recall and slows the search, the same speed-versus-recall dial you saw with `ef`.

IVF and HNSW side by side

	HNSW	IVF (for example IVFFlat)
Core idea	A layered graph you walk towards the query	Clusters of vectors, where you search only the nearest clusters
Main query knob	`ef`	Number of clusters probed
Main build choices	`m` and `ef_construct`	Number of clusters, chosen up front
Needs existing data to build?	No, vectors are added one by one	Yes, the clusters are learned from the data you already have

	HNSW	IVF (for example IVFFlat)
New data over time	Inserted straight into the graph	Goes into existing clusters, which can drift out of date, so periodic rebuilding is common
Recall at a given speed	Usually better, with less tuning	Good when tuned, but can slip as data grows
Memory	Higher, because of the graph links	Lower, because it stores little beyond the vectors
Build time	Slower	Faster

When is HNSW worth it?

These rules of thumb come from practical experience with high-dimensional embeddings, such as 3072 numbers per vector. Treat the figures as guidance, not laws:

- **At small scale, either index is fine.** With a few dozen to a few hundred vectors the index choice will not change your life. The real cost is memory, because a 3072-dimension vector alone takes 12 KB. Reducing the dimension, or storing vectors at lower precision, helps more than switching indexes.
- **HNSW starts to pay off at larger sizes.** The suggested tipping point was somewhere past roughly 30,000 to 50,000 vectors.
- **IVF needs more babysitting.** Its cluster count and probes need careful tuning, and recall can fall as the dataset grows. HNSW was described as closer to "set it and forget it".
- **HNSW costs memory and build time.** On a small server that matters, so check index size before switching.
- **Moving between them is possible but is real work:** you build the new index alongside the old one, test it, then drop the old one. Teams that expect to grow quickly often start on HNSW to avoid the migration.
- Newer **disk-based** indexes such as DiskANN exist for very large collections, and the field keeps moving.

Why HNSW is the right choice with Qdrant

It is Qdrant's index

Qdrant is built around HNSW for vector search, and it extends the graph so that filters on metadata, such as a tenant, still search efficiently. Using it means working with the database's best-supported path.

RAG data keeps changing

Documents arrive and are updated all the time. HNSW takes new vectors as they come, while IVF's clusters are learned from a snapshot and age as the data shifts.

No training step

You do not have to guess a cluster count before you have data. Fewer decisions up front means fewer ways to go wrong.

Strong recall, little tuning

You usually get high recall by adjusting a single query knob, `ef`, rather than juggling several.

Avoids a later migration

Rebuilding an index over a large collection is expensive. Starting with the one you expect to need is cheaper than switching later.

The cost is manageable

HNSW's extra memory can be offset with quantization and smaller dimensions.

tip

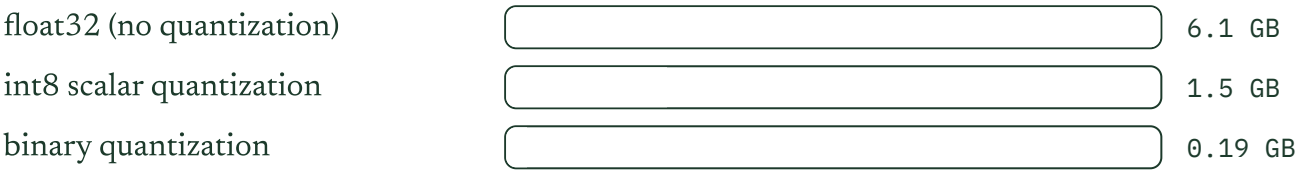
IVF still makes sense when memory is very tight, the data is mostly static, or your database only offers IVF. The skill to take away is not "HNSW always wins", but knowing the trade-off: **HNSW buys recall and convenience with memory, IVF saves memory and asks for more tuning.**

> *Go deeper* Pinecone: vector indexes in FAISS (flat, IVF, HNSW)
> pgvector: IVFFlat and HNSW indexing

Quantization

Vectors are expensive: 1536 numbers at 4 bytes each is about 6 KB per chunk, and a million chunks is about 6 GB. **Quantization** stores a compressed, lower-precision version instead.

float32 vector → Lower-precision representation → Smaller memory footprint



memory for 1 million vectors of 1536 dimensions, vectors only. Binary needs special care to keep quality.

Potential benefits

- Lower memory use
- Smaller storage
- Possible speed gains

Potential cost

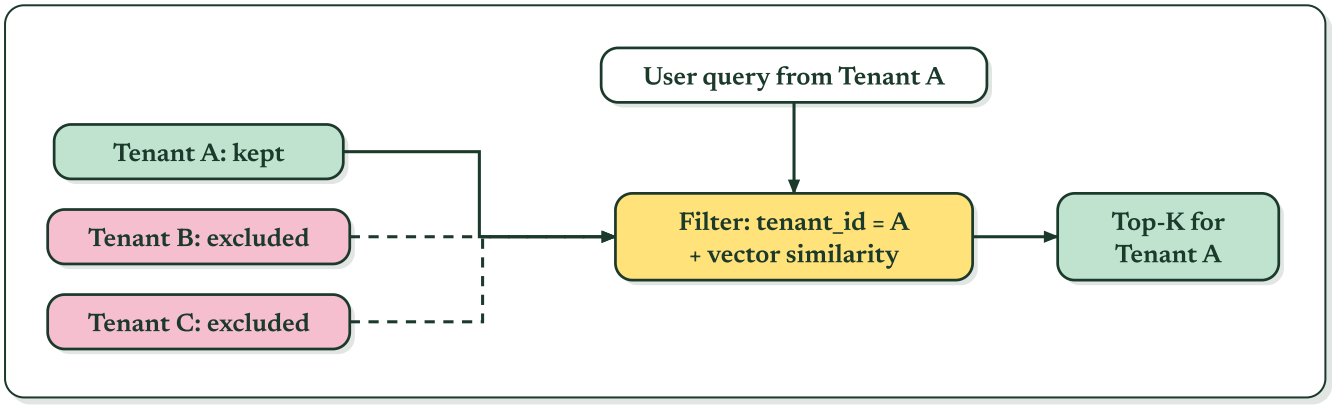
- Approximation, so possible quality loss

```
client.create_collection(
    collection_name="compressed",
    vectors_config=models.VectorParams(size=1536, distance=models.Distance.C
    quantization_config=models.ScalarQuantization(
        scalar=models.ScalarQuantizationConfig(
            type=models.ScalarType.INT8,   # float32 -> 8-bit integers
            always_ram=True,
        )
    ),
)
```

Qdrant supports scalar, product and binary quantization, described in its quantization guide and the article on binary quantization. Focus on *what problem it solves*, not the implementation.

Filtering with vector search

Similarity alone is not enough. Imagine three companies share one collection. A question from Tenant A must never retrieve Tenant B's documents, no matter how similar they are.



Copy

```
# 1. index the field you filter on, so filtered search stays fast
client.create_payload_index(
    collection_name="documents",
    field_name="tenant_id",
    field_schema=models.PayloadSchemaType.KEYWORD,
)

# 2. search with a filter
results = client.query_points(
    collection_name="documents",
    query=query_embedding[0],
    limit=5,
    query_filter=models.Filter(must=[
        models.FieldCondition(key="tenant_id", match=models.MatchValue(value=
    ]),
)
```

This is why metadata mattered back in Part two. See Qdrant's filtering guide.

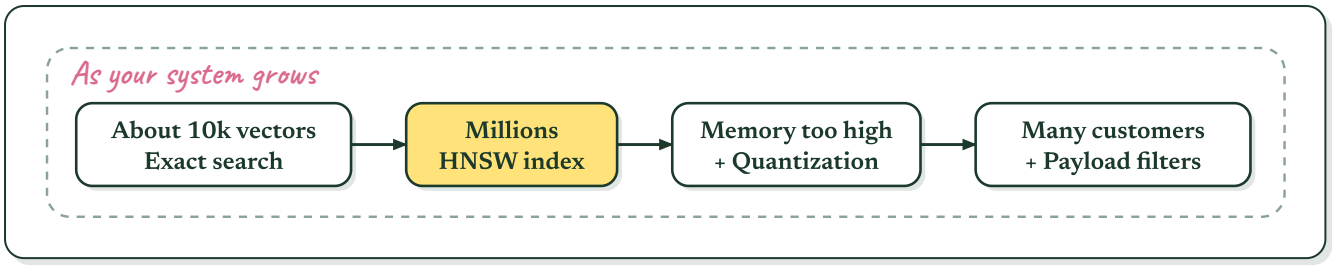
Comparing the search strategies

You have now met four tools: exact search, ANN with HNSW, quantization and filtering. They are not rivals. Some are **ways to find neighbours** (exact, HNSW) and some are **add-ons** that fix a specific problem (quantization, filters). Here is each one with its honest trade-offs.

Strategy	Pros	Cons
Exact / flat search	Perfect ranking. Nothing to tune. No index to build. The ground truth for evaluation.	Work grows with every vector. Too slow at millions of vectors.
ANN with HNSW	Fast at huge scale. Recall is tunable with `ef`. Qdrant's default index for vector search.	Approximate, so it can miss a true neighbour. The graph costs memory and build time.
Quantization	Much lower memory and storage. Can speed up search.	Lossy: recall can drop. Needs measuring before you trust it.
Filtering	Tenant isolation, permissions and metadata scoping. Required for any multi-user product.	Needs payload design up front and an index on filtered fields. Very selective filters can slow search.

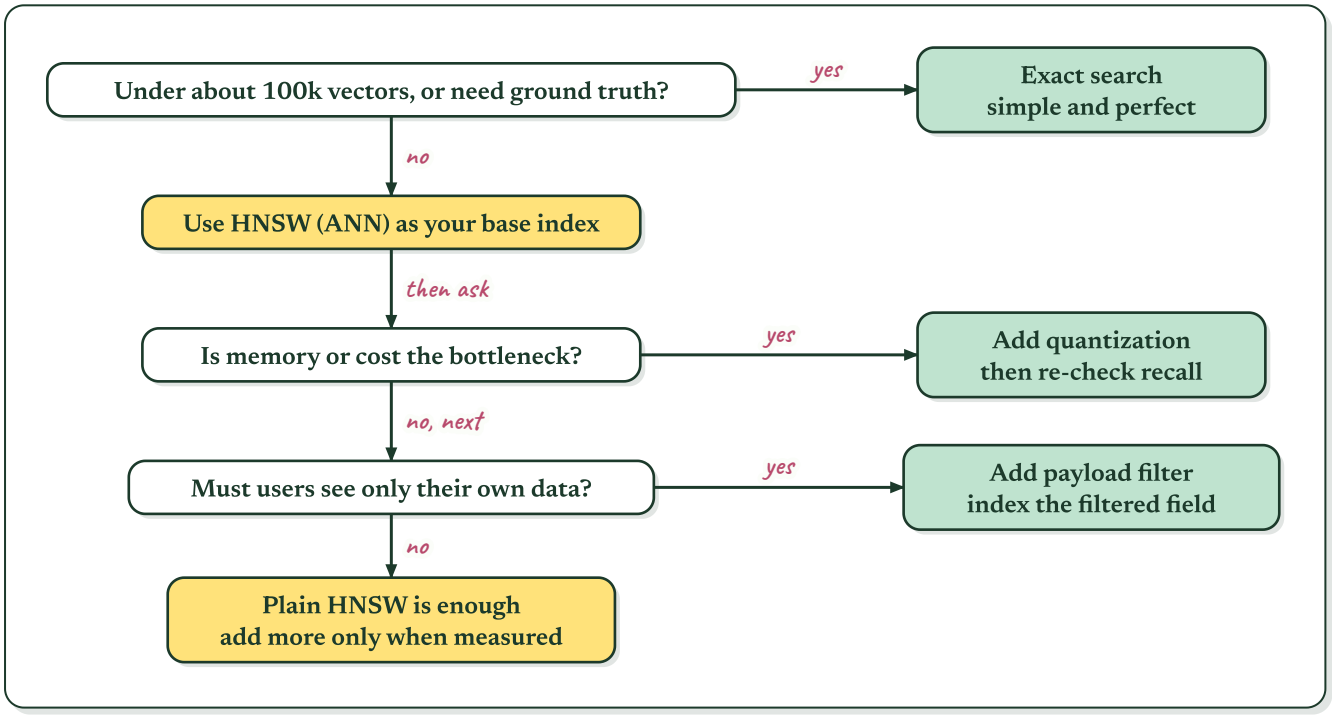
The growth path

Most real systems follow the same journey. You do not need to build the last box on day one. Add each step only when you can point to the problem it solves.



A decision guide with real cases

Use this when you are designing a new retrieval system. Start from the left column and only add what a measured problem demands.



Real-world cheat sheet

Scenario	Choose	Why
Small demo with 50 documents	Exact search	Tiny data, so perfect results cost nothing and there is no index to explain.

Scenario	Choose	Why
Measuring how good your retrieval really is	Exact search	It is the ground truth you compare approximate results against.
Support search across 20 million tickets, answers in milliseconds	HNSW	A full scan is too slow. A small recall loss is a fair price.
Image search over hundreds of millions of vectors on limited RAM	HNSW + quantization	Memory is the limit, and compressed vectors let the index fit.
Shared knowledge-base product serving many companies	Payload filter	Company A must never retrieve Company B's documents, however similar.
Chatbot over a small team wiki (about 5,000 chunks)	Exact search	At this size a full scan is still fast, so there is nothing to tune.

Measure recall instead of guessing

Because HNSW is approximate, check how much it misses. Run the same query as exact and approximate search, then count the overlap. This number is called **recall@K**.

```
K = 10

exact = client.query_points(
    collection_name="documents", query=query_embedding[0], limit=K,
    search_params=models.SearchParams(exact=True),
).points

approx = client.query_points(
    collection_name="documents", query=query_embedding[0], limit=K,
    search_params=models.SearchParams(hnsw_ef=64),
).points
```

```
recall = len({p.id for p in exact} & {p.id for p in approx}) / K
print(f"recall@{K} = {recall:.2f}")    # 1.00 means ANN found every true neig
```

If recall is too low, raise `hnsw_ef`. If latency is too high, lower it. The same check tells you whether quantization hurt quality.

> *Start with HNSW, measure recall against exact search, then add only what a measured problem demands.*
>
> Every extra feature adds cost and complexity. Let evidence, not fashion, decide which ones you keep.

part seven

Putting it together

The complete mental model

Here is the picture again, now with every box earned. You should be able to explain every arrow.

Indexing Documents → Parse → Chunk → Embed → Qdrant (vector + payload + index)

Querying User query → Query embedding → Vector search → Top-K → Prompt + context → LLM → Answer

Case study

Anthropic's Contextual Retrieval shows how much retrieval quality improves

Evaluation

Once you build real systems, measure them rather than trusting a few demo

when each stage is engineered carefully. The lesson: better RAG often comes from better *retrieval*, not a bigger LLM. Results are from their own setup, not universal guarantees.

questions. RAGAs covers retrieval relevance, context quality, faithfulness and answer quality.

Graph RAG

Vector retrieval finds semantic similarity. Graph retrieval adds entities, relationships and global structure. See Microsoft GraphRAG.

Enterprise RAG

Tenant isolation, retrieval security and data separation: Secure multitenant RAG.

Checklist

Tick these off as you go. Your progress is saved in this browser.

RAG and documents

- [ ] What RAG is, and why it is needed
- [ ] RAG vs LLM-only vs fine-tuning
- [ ] Indexing vs querying
- [ ] Parsing, cleaning and metadata
- [ ] Chunking, overlap and the chunk-size trade-off

Embeddings and similarity

- [ ] Explain what a dimension is, and why 1536 dimensions means 1536 numbers

- [ ] Describe the steps inside an embedding model
- [ ] Estimate how many dimensions cost in memory
- [ ] Generate embeddings with the OpenAI API
- [ ] Document vs query embeddings, and model compatibility
- [ ] Cosine, L2 and dot product, and which direction is "better"
- [ ] Rank documents and return Top-K by hand

Qdrant

- [ ] Collection, point, vector, payload
- [ ] In-memory mode, create collection, upsert, search
- [ ] KNN, Top-K, exact search, ANN
- [ ] HNSW and the m, ef_construct, ef knobs
- [ ] IVF as an alternative index, and how it compares with HNSW
- [ ] Filtering and quantization (concept)

Final self-test

Try to answer each question out loud before opening it.

▶ **What problem does RAG solve?**

▶ **Why do we chunk documents?**

▶ **What does "1536 dimensions" actually mean?**

▶ **Walk through what happens inside an embedding model.**

▶ **Why does one million 1536-dimension vectors need about 6 GB?**

▶ **Cosine vs L2: which direction is "more similar"?**

▶ **Why must query and document vectors come from the same model?**

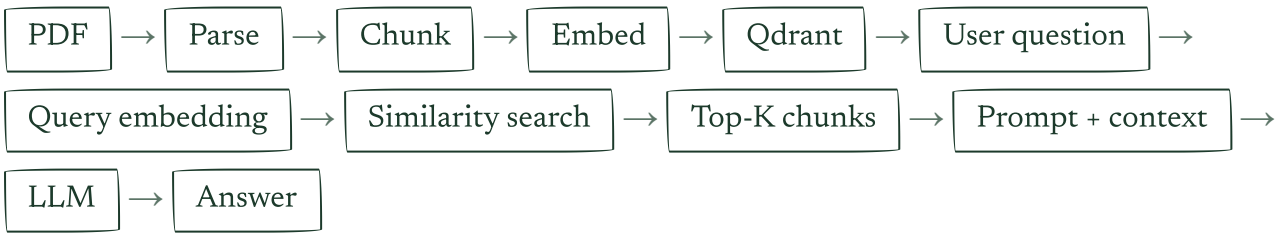
▶ **Why do vector indexes exist, and what is HNSW?**

▶ **How does IVF differ from HNSW, and why do we use HNSW with Qdrant?**

▶ **How does Qdrant improve on a naive full scan?**

Final challenge

Explain this entire pipeline in your own words, naming what happens at every arrow:



SHUTTLE

Want to go further? Explore the AI Engineering course from Coding Shuttle.